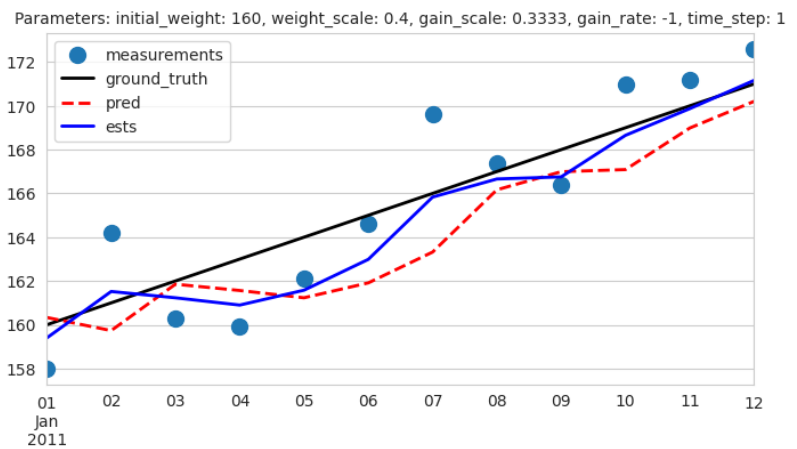
Example of Estimating Body Weight

- Two **problems**
  - The filter needs a correct guess of the weight change rate
  - The rate of change can vary over time
- **Solution**: estimate the rate of change from measurements
  - "Data is better than a guess, even if it's noisy"
  - Refine the estimate of the gain rate:

    new gain = old gain + 0.3 (measurement - prediction) / 1 day



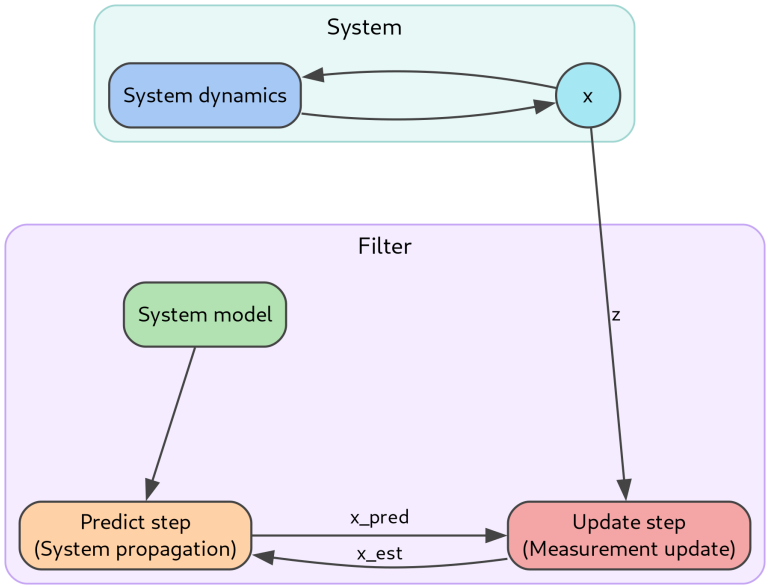
- The "state" is given by `weight` and `gain_rate`
  - You need to predict and update both

g-h Filter

- The previous algorithm is called **g-h filter**
  - g: scaling used to blend predicted state and measurement
  - h: scaling used to update the parameter of the system model based on the measurements
- If g is:
  - Smaller $\rightarrow$ follow more our model than the measurements
  - Larger $\rightarrow$ follow more the measurements than our model
  - Too large $\rightarrow$ follow the measurements and reject no noise
- If h is:
  - Smaller $\rightarrow$ Smoother but slower adaptation of the rate estimate
  - Larger $\rightarrow$ Faster adjustment of the estimated rate
- g-h filters have different values of g and h to achieve different properties
  - E.g., pick g to minimize the transient error when the derivative of the signal has a step (i.e., a discontinuity of the slope)
- Many filters (including Kalman filter) are just generalizations of a g-h filter

Control Theory Nomenclature

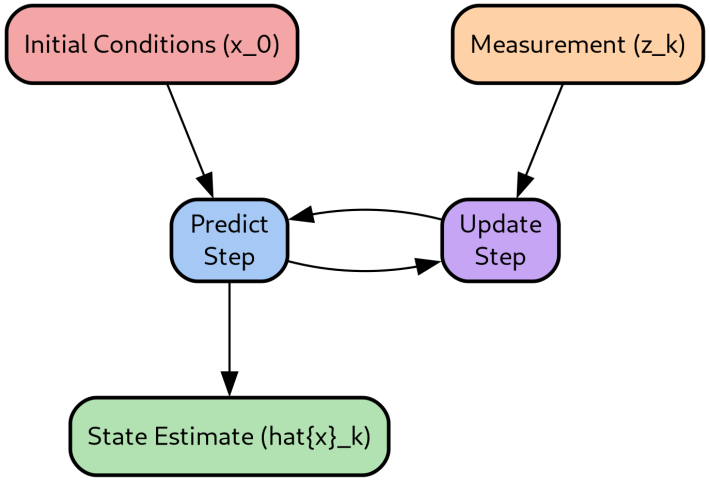
- State space models developed in control theory → different nomenclature
- **System**: object you want to estimate/track
- **Filter**: algorithm to estimate the state of the system
- **State of the system** `x`: current values you are interested in
  - Part of the state might be hidden



- **Measurement** `z`: the measured value of the system
  - Observable, but it can be inaccurate
- **State estimate** `x_est`: filter estimate of the state
- **System model**: mathematical model of the system
  - Typically there is error in the specification of the model
- **System propagation**: predict step using the system model to form a new state estimate `x_pred`
  - Because the system model and the measurements are imperfect, the estimate is imperfect
- **Measurement update**: update step

g-h Filter Algorithm: Pseudo-Code

1. **Initialization**
   - Initialize state of the filter
   - Initialize belief in the state
2. **Predict**
   - Use system model to predict state at next time step
   - Adjust belief to account for uncertainty in prediction
3. **Update**
   - Get measurement and associated belief about its accuracy
   - Use as estimate of the next state a point between estimated state and measurement



Interpretation of g

- If $g = 0$:
  - The filter follows the system model, ignoring the measurements
- If g increases:
  - The filter follows the measurements more, ignoring the prediction
  - Useful when measurements are accurate and the system model is inaccurate
- If $g = 1$:
  - The filter follows only the measurements, ignoring the system model

Interpretation of h

- You might need to estimate some model parameters from data, e.g.,
  - The change of weight
  - The rate of change of the measurements
  - The speed of the car on different terrains
- If $h = 0$:
  - The filter follows the previous values of the rate of change of the underlying model
  - I.e., it adapts slowly to the change of the signals
- If $h = 1$:
  - The filter reacts to the transient rapidly if the signal varies significantly with respect to the time step
- **Note**: an incorrect initial state (e.g., initial value/rate of change) is similar to a changing state

- ***One Dimensional Kalman Filters***

Updating Belief Using Gaussians

- The Bayes theorem tells that:

$$\text{posterior} = \text{normalized}(\text{prior} \times \text{likelihood})$$

- If the prior and the likelihood are Gaussian the result is also Gaussian (conjugate prior)
  - The belief and probability are represented as a Gaussian
  - We can encode the PDF in terms of mean and std dev
  - Updating belief is equivalent to sum and multiplication of Gaussians
- Algorithm:

```
# Create prior (using current estimate and system model)
prior = predict(x, process_model)

# Create likelihood (using measurement).
likelihood = gaussian(z, sensor_var)

# Update belief using prior and likelihood
posterior = update(prior, likelihood)
```

Sum of Gaussians

- The sum of two independent Gaussians

$$Normal(\mu_1, \sigma_1^2)$$
$$Normal(\mu_2, \sigma_2^2)$$

  is a Gaussian $Normal(\mu, \sigma^2)$ with:

$$\mu = \mu_1 + \mu_2$$
$$\sigma^2 = \sigma_1^2 + \sigma_2^2$$

- The mean is the sum of the mean (by linearity)
- The variance always increases

Product of Gaussians

- The product of two independent Gaussians

$$Normal(\mu_1, \sigma_1^2)$$
$$Normal(\mu_2, \sigma_2^2)$$

is a Gaussian $\mathrm{N}(\mu, \sigma^2)$ with:

$$\mu = \frac{\mu_1\sigma_2^2 + \mu_2\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$
$$\sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

- **Interpretation:**
  - The variance may be reduced as more information is incorporated
  - If one Gaussian N_1 is much narrower than the other (i.e., one measure is more accurate), the result is pushed towards N_1
  - If two Gaussians are similar (i.e., two measures corroborate each other), the result becomes more certain

Kalman Gain

- Assume that:
  - x is the model prediction
  - z indicates the measurements
- The mean of the posterior is:

$$\mu = \frac{\sigma_x^2 \mu_z + \sigma_z^2 \mu_x}{\sigma_x^2 + \sigma_z^2} = \frac{\sigma_x^2}{\sigma_x^2 + \sigma_z^2}\mu_z + \frac{\sigma_z^2}{\sigma_x^2 + \sigma_z^2}\mu_x = K\mu_z + (1-K)\mu_x$$

- The Kalman Gain K:
  - Is the scaling term that mixes the prediction and the measurement
  - Depends on the ratio of uncertainty of prior and measurement